

Proyecto Deep Learning: Detección de Imágenes Generadas por IA utilizando CNNs

Camilo Benavides Ramírez, Brayan Santiago Ramírez Silva, Mariana Velásquez Velásquez
Universidad de Antioquia — Facultad de Ingeniería
Informe Final — Entrega 2

Resumen—Este informe documenta el desarrollo de un sistema de visión computacional, basado en una Red Neuronal Convolutiva (CNN), para clasificar imágenes como reales o generadas por Inteligencia Artificial, empleando el dataset CIFAKE. La solución implementa el ciclo de vida completo del dato —ingesta, preprocesamiento, modelado, entrenamiento y evaluación— estructurado en notebooks numerados y reproducibles en Google Colab. La arquitectura baseline, una CNN secuencial con bloques convolucionales, reducción de dimensionalidad y regularización por *dropout*, alcanzó una exactitud del 92.87% y un F1-Score macro de 0.93 sobre 20,000 imágenes de prueba, estableciendo una línea de base sólida para futuras optimizaciones.

Index Terms—Deep Learning, Redes Neuronales Convolucionales, CNN, CIFAKE, Imágenes Sintéticas, Clasificación Binaria.

I. INTRODUCCIÓN

La Inteligencia Artificial Generativa ha avanzado de forma acelerada durante la última década. Los modelos de difusión de texto a imagen —como Stable Diffusion, Midjourney o DALL-E— han democratizado la creación de contenido visual hiperrealista, hasta el punto de dificultar la distinción entre una fotografía capturada por un sensor óptico y una matriz de píxeles generada estadísticamente.

Esta capacidad introduce vectores de riesgo en sectores críticos: la ciberseguridad y la verificación de identidad (donde las pruebas de vida pueden ser vulneradas), el periodismo y la integridad de la información (con la proliferación de noticias falsas acompañadas de evidencia visual fabricada) y la moderación de contenido en redes sociales (que requiere mecanismos automáticos para etiquetar material sintético). Ante este panorama, desarrollar sistemas capaces de auditar la procedencia de una imagen se ha vuelto una necesidad imperativa.

El objetivo de este proyecto es diseñar, entrenar y evaluar un modelo de aprendizaje profundo basado en CNN que reciba una imagen digital y emita una predicción probabilística sobre su naturaleza, clasificándola como **REAL** o **FAKE**. En esta iteración, la meta concreta es consolidar un *baseline* funcional que resuelva el flujo de extremo a extremo de manera reproducible.

II. ESTRUCTURA DE LOS NOTEBOOKS ENTREGADOS

La solución se organiza en cuatro notebooks numerados secuencialmente, cada uno autónomo y ejecutable en Google Colab sin errores. La carga de datos se incluye dentro de

cada notebook (mediante kagglehub) para garantizar la reproducibilidad independiente de cada uno.

- **01 - exploración de datos.ipynb**: ingestión del dataset CIFAKE desde Kaggle con `kagglehub`, carga de los conjuntos de entrenamiento y prueba con `image_dataset_from_directory`, inspección de la estructura (clases y número de lotes) y visualización de muestras representativas de cada clase.
- **02 - preprocesado.ipynb**: normalización de los tensores al rango $[0, 1]$ (división por 255.0) y configuración de la canalización de datos con `cache()` y `prefetch(AUTOTUNE)`. Incluye una verificación del rango de píxeles resultante.
- **03 - arquitectura de línea de base.ipynb**: definición de la CNN secuencial (bloques `Conv2D`, `MaxPooling2D`, `Dropout` y `Dense`), compilación con optimizador Adam y pérdida `binary_crossentropy`, entrenamiento durante 10 épocas, graficado de las curvas de aprendizaje y guardado del modelo entrenado (`modelo_cifake.keras`).
- **04 - evaluacion_y_resultados.ipynb**: carga del modelo entrenado (o entrenamiento automático de respaldo si no se encuentra el archivo), evaluación sobre el conjunto de prueba, matriz de confusión, reporte de clasificación (precision, recall, F1) y curva ROC.

Este diseño modular separa las responsabilidades del flujo de trabajo y facilita la trazabilidad: cada notebook puede ejecutarse de forma aislada para inspeccionar una etapa específica del proceso.

III. DESCRIPCIÓN DE LA SOLUCIÓN

A. Ingesta de datos

La ingestión se realiza con la librería `kagglehub`, que descarga el dataset `birdy654/cifake-real-and-ai-generated-synthetic-images` directamente desde Kaggle y devuelve la ruta local donde queda almacenado en caché. Esta descarga se ejecuta dentro del propio notebook, por lo que no requiere subir manualmente los archivos ni montar Google Drive.

A partir de la ruta obtenida se construyen los `tf.data.Dataset` de entrenamiento y prueba mediante `image_dataset_from_directory`, fijando un tamaño de imagen de 32×32 , un tamaño de lote de 32 y el orden de clases `['FAKE', 'REAL']` (de modo que `FAKE = 0` y `REAL = 1`), con etiquetas en modo binario.

El corpus total consta de 120,000 imágenes RGB de 32×32 píxeles, perfectamente balanceadas: 100,000 para entrenamiento (50,000 reales y 50,000 sintéticas) y 20,000 para prueba (10,000 y 10,000). Este balance 50/50 evita sesgos algorítmicos y simplifica la interpretación de las métricas.

1) *Cómo obtener y hacer disponibles los datos:* La obtención de los datos está integrada en los notebooks y no requiere descargas ni cargas manuales. El procedimiento para reproducir cualquier notebook en Google Colab es el siguiente:

- 1) **Instalación de la librería:** cada notebook ejecuta `!pip install -q kagglehub` en su primera celda.
- 2) **Descarga automática:** la instrucción `kagglehub.dataset_download(...)` descarga el dataset desde Kaggle y devuelve la ruta local (path) donde queda almacenado en caché. La primera ejecución descarga aproximadamente 200 MB; las posteriores reutilizan la caché.
- 3) **Acceso a las particiones:** dentro de la ruta descargada, las imágenes están organizadas en las carpetas `train/` y `test/`, y dentro de cada una en subcarpetas `REAL/` y `FAKE/`, que `image_dataset_from_directory` interpreta directamente como etiquetas.
- 4) **Autenticación (si aplica):** al tratarse de un dataset público, normalmente no se requieren credenciales. Si el entorno solicita autenticación de Kaggle, basta con seguir las indicaciones en pantalla o cargar el archivo `kaggle.json` de la cuenta del usuario.

Gracias a este enfoque, cualquier evaluador puede abrir un notebook en Colab, ejecutar todas las celdas y obtener los datos de forma autónoma.

B. Preprocesamiento

El preprocesamiento es deliberadamente simple y consta de dos componentes:

- **Normalización a escala $[0, 1]$:** cada píxel se divide por 255.0. Llevar los valores del rango $[0, 255]$ al rango $[0, 1]$ estabiliza el descenso del gradiente y acelera la convergencia del entrenamiento.
- **Optimización de la canalización:** se aplica `cache()`, que mantiene los datos en memoria tras la primera época, y `prefetch(tf.data.AUTOTUNE)`, que prepara el siguiente lote mientras el actual se procesa, reduciendo los cuellos de botella de entrada/salida.

No se aplican técnicas de aumento de datos (*data augmentation*) en esta iteración, con el fin de mantener el *baseline* lo más limpio y reproducible posible.

C. Arquitectura de la CNN

El modelo se implementa con la API secuencial de Keras (TensorFlow) y está diseñado para extraer jerárquicamente características espaciales de las imágenes:

- 1) **Bloques convolucionales:** tres capas `Conv2D` con 32, 64 y 128 filtros (kernel 3×3 , activación ReLU) que detectan patrones locales —bordes, texturas y el “ruido” residual característico que dejan los modelos de difusión.

- 2) **Reducción de dimensionalidad:** dos capas `MaxPooling2D` (2×2) que reducen la resolución espacial de los mapas de características y aportan invariancia a la traslación.
- 3) **Aplanamiento y capa densa:** una capa `Flatten` seguida de una capa `Dense` de 128 neuronas con activación ReLU.
- 4) **Regularización:** una capa `Dropout` con tasa de 0.5 (50%) antes de la salida, técnica fundamental para mitigar el sobreajuste en un dataset de 100,000 imágenes.
- 5) **Clasificador final:** una capa `Dense` de una sola neurona con activación `sigmoid`, que produce una probabilidad en $[0, 1]$ interpretable como pertenencia a la clase positiva.

El modelo se compila con el optimizador **Adam**, la función de pérdida `binary_crossentropy` (apropiada para clasificación binaria) y la métrica de exactitud (*accuracy*).

IV. DESCRIPCIÓN DE LAS ITERACIONES

Esta primera iteración se centró deliberadamente en lograr el ciclo de vida completo del dato de extremo a extremo (*end-to-end*) antes que en maximizar el rendimiento. El propósito fue validar que cada eslabón del flujo —ingesta, normalización, definición de la arquitectura, entrenamiento y evaluación— funciona correctamente y de forma reproducible, estableciendo así un *baseline* confiable sobre el cual iterar.

El entrenamiento se ejecutó durante 10 épocas. El modelo demostró ser computacionalmente ligero gracias a la baja dimensionalidad de las imágenes ($32 \times 32 \times 3$): la primera época requirió alrededor de 287 segundos (inicialización y construcción de la caché) y las nueve restantes se estabilizaron en 43–45 segundos cada una, para un tiempo total inferior a 15 minutos en hardware estándar.

La función de pérdida convergió de forma consistente, descendiendo desde 0.3467 hasta 0.1057 en entrenamiento, lo que evidencia un aprendizaje efectivo y estable sin signos severos de sobreajuste, en buena medida gracias a la regularización por *dropout*.

V. DESCRIPCIÓN DE LOS RESULTADOS

A. Métricas de evaluación

Dado el balanceo perfecto del dataset, las métricas se interpretan de forma directa. Considerando VP, VN, FP y FN como verdaderos/falsos positivos y negativos, las métricas empleadas son:

$$\text{Accuracy} = \frac{VP + VN}{VP + VN + FP + FN} \quad (1)$$

$$\text{Precision} = \frac{VP}{VP + FP}, \quad \text{Recall} = \frac{VP}{VP + FN} \quad (2)$$

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad F1_{\text{macro}} = \frac{F1_{\text{REAL}} + F1_{\text{FAKE}}}{2} \quad (3)$$

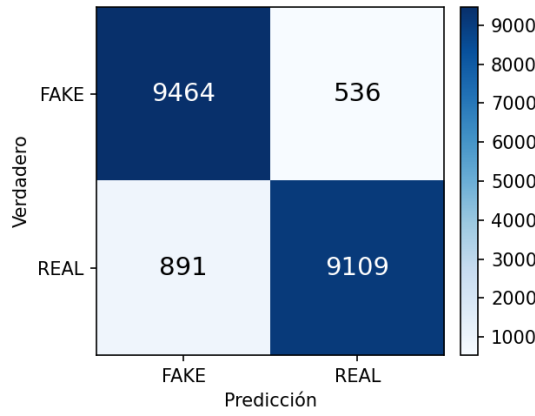


Figura 1. Matriz de confusión sobre el conjunto de prueba (20,000 imágenes).

Sobre el conjunto de prueba de 20,000 imágenes, el modelo baseline obtuvo los resultados globales resumidos en la Tabla I.

Cuadro I
MÉTRICAS GLOBALES SOBRE EL CONJUNTO DE PRUEBA.

Métrica	Valor
Exactitud (Accuracy)	92.87%
F1-Score macro	0.93
Recall (FAKE)	0.95
Precision (REAL)	0.94
Pérdida de validación	0.2324

B. Análisis de la matriz de confusión

La matriz de confusión obtenida sobre las 20,000 imágenes de prueba (Fig. 1) resume el comportamiento del clasificador. De aquí se desprende que:

- **Clase FAKE:** el modelo identificó correctamente 9464 de 10,000 imágenes sintéticas, alcanzando un *recall* de 0.95. Solo se le escapa cerca del 5% de las imágenes generadas por IA (falsos negativos).
- **Clase REAL:** se clasificaron correctamente 9109 de 10,000 imágenes reales, con una *precision* cercana a 0.94. Esto garantiza que, cuando el modelo etiqueta una imagen como REAL, lo hace con alto grado de certeza, minimizando el riesgo de censurar contenido genuino.

El F1-Score macro de 0.93 confirma un equilibrio excepcional entre ambas clases. La curva ROC complementa este análisis mostrando una elevada capacidad discriminativa (área bajo la curva alta), coherente con las métricas anteriores.

C. Interpretación

Los resultados son sobresalientes para una primera iteración: el modelo converge rápidamente y generaliza bien sobre datos no vistos. El alto *recall* en la clase FAKE es deseable en moderación de contenido y verificación, mientras que la alta *precision* en la clase REAL protege el contenido legítimo de los usuarios.

VI. REVISIÓN DE LITERATURA

La detección de imágenes generadas por inteligencia artificial ha sido abordada recientemente desde distintos enfoques, incluyendo clasificadores supervisados, detectores con capacidad de generalización entre generadores y métodos orientados a modelos de difusión. En la literatura, la evaluación suele apoyarse en métricas como *accuracy*, *precision*, *recall*, F1-score, *average precision* (AP) y AUC-ROC. La Tabla II resume algunos trabajos representativos.

Cuadro II
RESUMEN DE TRABAJOS RELACIONADOS EN DETECCIÓN DE IMÁGENES GENERADAS POR IA.

Trabajo	Aporte principal	Métricas
Bird y Lotfi [2]	Proponen el dataset CIFAKE y abordan el problema como clasificación binaria entre imágenes reales y sintéticas.	Accuracy, Precision, Recall, F1
Wang et al. [3]	Detección de imágenes generadas por CNN y generalización hacia generadores no vistos.	AP, Accuracy
Ojha et al. [4]	Detectores con énfasis en la generalización entre familias de modelos generativos.	AP, Accuracy
Cazenavette et al. [5]	FakeInversion: detección de imágenes de modelos de texto a imagen no vistos en el entrenamiento.	AUC-ROC, AP, Accuracy

El presente trabajo se alinea con la línea de Bird y Lotfi [2], adoptando CIFAKE como banco de pruebas y la clasificación binaria como formulación, y deja como trabajo futuro la evaluación de generalización entre generadores explorada por [3], [4], [5].

VII. CONCLUSIONES Y TRABAJO FUTURO

El proyecto cumplió su objetivo principal: se construyó un sistema *end-to-end* reproducible que clasifica imágenes reales y generadas por IA con una exactitud del 92.87% y un F1-Score macro de 0.93. La CNN secuencial, pese a su simplicidad, demostró ser un *baseline* robusto y computacionalmente eficiente. Para futuras iteraciones se plantean las siguientes líneas de mejora: aumento de datos (*data augmentation*), normalización por lotes (*Batch Normalization*), arquitecturas más profundas o transferencia de aprendizaje, y evaluación de la generalización frente a generadores distintos a Stable Diffusion v1.4.

REFERENCIAS

- [1] J. J. Bird y A. Lotfi, "CIFAKE: Real and AI-Generated Synthetic Images," Kaggle, 2024. [Online]. Disponible: <https://www.kaggle.com/datasets/birdy654/cifake-real-and-ai-generated-synthetic-images>
- [2] J. J. Bird y A. Lotfi, "CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images," arXiv preprint arXiv:2303.14126, 2023.
- [3] S.-Y. Wang, O. Wang, R. Zhang, A. Owens y A. A. Efros, "CNN-Generated Images Are Surprisingly Easy to Spot... for Now," CVPR, 2020, pp. 8695–8704.
- [4] U. Ojha, Y. Li y Y. J. Lee, "Towards Universal Fake Image Detectors That Generalize Across Generative Models," CVPR, 2023, pp. 24480–24489.
- [5] G. Cazenavette, A. Sud, T. Leung y B. Usman, "FakeInversion: Learning to Detect Images from Unseen Text-to-Image Models by Inverting Stable Diffusion," CVPR, 2024, pp. 10759–10769.